পর্ব ৬ · ক্লাস ২৮

ল্যাপটপ থেকে সার্ভার পর্যন্ত

ডিপ্লয়মেন্টের মৌলিক: **prod.py** · পরিবেশ চলক · Gunicorn · WhiteNoise · PostgreSQL · রিহার্সাল

IMS · ক্লাস ২৮ · স্লাইড ১–১৯

শুরু

সার্ভারে `runserver` কেন চলবে না?

কাজ করে, কিন্তু দায়িত্ব নেয় না

১. এক থ্রেড, এক অনুরোধ

ডেভ সার্ভার অনুরোধের পর অনুরোধ লম্বা লাইনে সার্ভ করে। ৩০ জন শিক্ষক একসাথে মার্কস এন্ট্রি দিলে সবাই অপেক্ষা করবে।

২. নিজেই বলে "প্রোডাকশনে চালাবেন না"

`runserver` চালু হলেই সতর্কবার্তা দেয়; নিরাপত্তা হার্ডেনিং, স্ট্যাটিক ফাইল, লগ — কোনোটাই এর কাজ নয়।

৩. DEBUG=True মানে সব গোপন তথ্য পর্দায়

একটি এরর হলেই সেটিংস, SQL, ফাইল-পথ — যা কিছু দরকার। শিক্ষকের ব্রাউজারে সেটি দেখানো মানে সেটি ফাঁস করা।

তাই ডিপ্লয়মেন্ট মানে কেবল "অন্য কম্পিউটারে চালানো" নয় — **অন্য একটি ভঙ্গি**: কঠোর সেটিংস, সত্যিকারের অ্যাপ সার্ভার, আসল ডেটাবেস, আর পেছনে ওয়েব সার্ভার।

শিক্ষকের নোট: প্রথম প্রশ্ন করুন: "আপনার প্রজেক্ট সার্ভারে দিলে কী হবে?" বেশিরভাগ উত্তর "চলবে" — আসল উত্তর "চলবে, কিন্তু ভাঙবে"।

মানচিত্র

তিনটি পরিবেশ, তিনটি উদ্দেশ্য

IMS-এ তিনটি সেটিংস ফাইল আছে — একটি নয়

দিক	dev	`sqlite` (ডেমো)	prod
DEBUG	True	True	**False**
ডেটাবেস	PostgreSQL	SQLite ফাইল	PostgreSQL
স্ট্যাটিক	ডেভ সার্ভার	ডেভ সার্ভার	WhiteNoise (সংকুচিত)
ইমেইল	কনসোলে ছাপে	কনসোলে ছাপে	SMTP
ALLOWED_HOSTS	`*`	`*`	আসল ডোমেইন, `*` নিষিদ্ধ
HTTPS	নেই	নেই	রিডাইরেক্ট + HSTS
কমান্ড	`runserver`	`run.sh`	`gunicorn`

তিনটি ফাইলই `base.py` থেকে উত্তরাধিকার নেয় — শুধু পার্থক্যটুকু ওভাররাইড করে। `from .base import *`

একটি ফাইলে `if DEBUG:` লিখে সব সামলানো যায় না — একটি ভুল মানেই প্রোডাকশনে ডিবাগ চালু।

শিক্ষকের নোট: বলুন: "ডেমো আর প্রোডাকশনের মধ্যে তফাত সেটিংসে, কোডে নয়।" একই কোড তিনটি ভঙ্গিতে চলে — এটাই সেটিংস-স্প্লিটের পুরো লাভ।

প্রথম নিয়ম

`prod.py` নিজেই থামে, ভুল মান নিলে

দুইটি রক্ষী — সত্যি চালিয়ে দেখা আউটপুট

```
# config/settings/prod.py
DEBUG = False

if SECRET_KEY == DEV_INSECURE_KEY:
    raise RuntimeError(
        "Refusing to start: SECRET_KEY is the "
        "development default. Generate one with: "
        "python -c \"import secrets;"
        " print(secrets.token_urlsafe(50))\""
    )

if not ALLOWED_HOSTS or "*" in ALLOWED_HOSTS:
    raise RuntimeError(
        "ALLOWED_HOSTS must list real hostnames "
        "in production; '*' is not allowed."
    )
```

চালিয়ে পাওয়া ফল

`RuntimeError: Refusing to start: SECRET_KEY …` ·
`RuntimeError: ALLOWED_HOSTS must list real hostnames…`

ডিজাইনের সিদ্ধান্ত

সার্ভার চালু হওয়ার আগেই ব্যর্থ — ভাঙা সাইট চালু থাকার চেয়ে বন্ধ থাকা ভালো।

একই কৌশল

`DEBUG` ডিফল্ট `False`; ভুলে চালু রাখা প্রায় অসম্ভব।

দর্শন

"ডিপ্লয় কাজ করাতে সেটিং শিথিল করবেন না — কারণ ঠিক করুন, সেটিং নয়।"

শিক্ষকের নোট: ক্লাসে ইচ্ছাকৃতভাবে পুরোনো ডেভ কী দিয়ে চালান — লম্বা এররটি পর্দায় দেখান। "সেটিং নিজে নিজেকে রক্ষা করছে।"

প্রমাণ

`check --deploy` — এক লাইনের নিরীক্ষা

আজ সত্যি চালানো: কোনো সমস্যা নেই

```
# ১. আসল কী বানান
python -c "import secrets;
print(secrets.token_urlsafe(50))"

# ২. প্রোডাকশন সেটিংসে চালান
SECRET_KEY="…" \
ALLOWED_HOSTS="ims.example.com" \
DJANGO_SETTINGS_MODULE=config.settings.prod \
python manage.py check --deploy

# আউটপুট:
System check identified no issues (0 silenced).
```

এটি কী দেখে

HSTS · SSL রিডাইরেক্ট · সিকিউর কুকি · `X-Frame-Options` · `DEBUG` · সিক্রেট · হোস্ট

কেন নিজে না করে

এটি ডিপ্লয়ের **সর্বনিম্ন** শর্ত — পাস করলেই নিরাপদ নয়, কিন্তু ফেল করলে অবশ্যই অনিরাপদ।

কোথায় চালাবেন

লোকালে rehearsel-এ **এবং** সার্ভারে ডিপ্লয়ের পরপরই — দুই জায়গায়।

শিক্ষকের নোট: "0 silenced" কথাটি ব্যাখ্যা করুন — কোনো চেক নীরবে চাপা দেওয়া হয়নি; সব চেক চলে, সব পাস করেছে।

কনফিগারেশন

প্রতিটি পরিবর্তনশীল মান = পরিবেশ চলক

কোডে কোনো পাসওয়ার্ড নয়, কোনো ডোমেইন নয়

```
# config/settings/base.py
def env(key, default=None):
    value = os.environ.get(key, default)
    if value is None:
        raise RuntimeError(
            f"Missing required environment "
            f"variable: {key}. Copy .env.example "
            f"to .env and fill it in.")
    return value

def env_bool(key, default=False):
    return env(key, str(default)).strip().lower() \
        in {"1", "true", "yes", "on"}

def env_list(key, default=""):
    raw = env(key, default)
    return [x.strip() for x in raw.split(",") if x.strip()]
```

চলক	থাকে কোথায়
`SECRET_KEY`	হোস্টের সিক্রেট স্টোর
`ALLOWED_HOSTS`	প্রতি পরিবেশে আলাদা
`CSRF_TRUSTED_ORIGINS`	আসল https ডোমেইন
`DB_NAME / DB_USER / DB_PASSWORD / DB_HOST`	ডেটাবেস প্রদানকারী
`EMAIL_HOST / USER / PASSWORD`	মেইল সেবা
`LOG_LEVEL / SECURE_SSL_REDIRECT`	অপারেটরের সিদ্ধান্ত

`.env` কখনো Git-এ নয় — `.env.example` যায়, আসলটি যায় না। ফাঁস হওয়া কী = ফাঁস হওয়া অ্যাপ।

শিক্ষকের নোট: `env()`-এর কাজ শুধু পড়া নয় — **না থাকলে স্পষ্ট বার্তা দিয়ে থামা**। নীরব ডিফল্ট সবচেয়ে বিপজ্জনক।

গোপনীয়তা

`SECRET_KEY` ফাঁস হলে কী ভাঙে

এটি পাসওয়ার্ড নয় — এটি অ্যাপের হস্তাক্ষর

সেশন কুকি

কী দিয়ে সেশন স্বাক্ষরিত — কী জানলে যে কেউ বৈধ সেশন কুকি বানাতে পারে।

পাসওয়ার্ড রিসেট টোকেন

টোকেনের স্বাক্ষরে এই কী — কী জানলে যেকোনো অ্যাকাউন্টের রিসেট লিংক বানানো যায়।

CSRF টোকেন

ফর্মের গোপন মান — কী জানলে ক্রস-সাইট অনুরোধে অনুকরণ সহজ।

বানানো: `python -c "import secrets; print(secrets.token_urlsafe(50))"` — ৫০+ অক্ষর, প্রতিটি পরিবেশে আলাদা।

ফাঁস হলে: কী পাল্টান → সব সেশন বাতিল হবে (সবাই আবার লগইন করবে), রিসেট লিংকের পুরোনো টোকেনও অচল।

শিক্ষকের নোট: জোর দিন: এই কী রোটেট করলে ব্যবহারকারীদের লগআউট হয়ে যায় — সেটি দোষ নয়, নিরাপত্তার দাম। রাতে করুন, ব্যবহার কম থাকলে।

ঠিকানা

`ALLOWED_HOSTS` ও `CSRF_TRUSTED_ORIGINS`

দুটি আলাদা দরজা, দুটি আলাদা আক্রমণ

সেটিং	কোন আক্রমণ থামায়	ভুল হলে লক্ষণ
`ALLOWED_HOSTS`	Host হেডারের জালিয়াতি — অন্য ডোমেইনের নামে অনুরোধ পাঠিয়ে পাসওয়ার্ড-রিসেট লিংক বিষাক্ত করা	অচেনা হোস্টে **400 Bad Request**
`CSRF_TRUSTED_ORIGINS`	অন্য সাইট থেকে ফর্ম সাবমিট — লগইন থাকা ব্রাউজার দিয়ে	https ডোমেইনে সাবমিট করলে **403 CSRF**
`SECURE_PROXY_SSL_HEADER`	প্রক্সির পেছনে থাকা অ্যাপকে "এটি https" বোঝানো	অসীম **রিডাইরেক্ট লুপ**

IMS-এ: `ALLOWED_HOSTS="*"` হলে `prod.py` চালুই হয় না — এই ফাঁদটা কাঠামোগতভাবে বন্ধ।

বাস্তব ফল: সাবডোমেইন বা কাস্টম ডোমেইন যোগ করলে এই দুটি তালিকা **দুই জায়গায়** আপডেট করতে হয় — নইলে 400/403।

শিক্ষকের নোট: প্রশ্ন: "আইপি দিয়ে টেস্ট করছি, ডোমেইন নেই — তখন কী?" উত্তর: হোস্টের দেওয়া ডোমেইনটাই লিখুন; `*` দিয়ে "পরে ঠিক করব" সবচেয়ে সাধারণ ও সবচেয়ে বিপজ্জনক অভ্যাস।

HTTPS

সাতটি সুইচ একসাথে

`prod.py`-তে যা ইতিমধ্যেই চালু

সেটিং	মান	কার কাজ
`SECURE_SSL_REDIRECT`	True	http → https
`SECURE_HSTS_SECONDS`	2592000 (৩০ দিন)	ব্রাউজারকে বলে: এই সাইটে আর কখনো http নয়
`SECURE_HSTS_PRELOAD`	True	ব্রাউজার-তালিকায় স্থায়ী এন্ট্রি
`SESSION_COOKIE_SECURE`	True	সেশন কুকি কেবল https-এ যাবে
`CSRF_COOKIE_SECURE`	True	একই নিয়ম CSRF কুকিতে
`SESSION_COOKIE_HTTPONLY`	True	জাভাস্ক্রিপ্ট কুকি পড়তে পারবে না
`X_FRAME_OPTIONS`	DENY	অন্য সাইটে iframe-এ বসানো যাবে না

HSTS-এর ফাঁদ: একবার ব্রাউজার এই হেডার পেলে ৩০ দিন https-ই দাবি করবে। সার্টিফিকেট না থাকলে সাইট **খুলবেই না** — তাই TLS আগে, HSTS পরে।

শিক্ষকের নোট: প্রথম দিনেই `preload` জমা দেবেন না — ৩০ দিন চালু রেখে সমস্যা নেই দেখে তারপর। ভুল হলে ফেরানো কঠিন, কারণ ব্রাউজারে হেডার জমে থাকে।

স্ট্যাটিক

`collectstatic` ও WhiteNoise

৫টি ফাইল, ৫২৮ কিলোবাইট — প্রোডাকশনে কে পরিবেশন করবে?

```
python manage.py collectstatic --noinput
# static/ -> staticfiles/
#   ims.css
#   vendor/bootstrap.min.css
#   vendor/bootstrap-icons.css
#   vendor/bootstrap.bundle.min.js
#   vendor/fonts/bootstrap-icons.woff2

# prod.py
MIDDLEWARE = ["whitenoise.middleware."
              "WhiteNoiseMiddleware"] + MIDDLEWARE
STORAGES = {
  "staticfiles": {"BACKEND": "whitenoise.storage."
                  "CompressedManifestStaticFilesStorage"},
}
WHITENOISE_MAX_AGE = 60 * 60 * 24 * 30
```

CompressedManifest কী করে

ফাইলের নামে হ্যাশ যোগ করে (`ims.7f3c1a.css`) — ৩০ দিন ক্যাশ করা নিরাপদ।

Manifest-এর ফাঁদ

টেমপ্লেটে `{% static "x.css" %}` না লিখে হার্ডকোড করলে `ValueError: Missing staticfiles manifest entry`।

WhiteNoise কেবল static

লোগো ও আপলোড করা `media/` এখান দিয়ে যায় না — আলাদা ভলিউম বা nginx দরকার।

কেন গুনছি ৫টি

CDN নেই — নিজের ৫টি ফাইলই যথেষ্ট। কমানো ফাইল-সংখ্যা = কম ভাঙার জায়গা।

শিক্ষকের নোট: `media/` ভুলে যাওয়া সবচেয়ে সাধারণ ডিপ্লয়-বাগ: রিডিপ্লয়ে লোগো মুছে যায়। ক্লাসে বলুন: "কোড আর ডেটা আলাদা — আপলোড করা ফাইল কোডের সঙ্গে যায় না।"

অ্যাপ সার্ভার

Gunicorn — `runserver`-এর জায়গা নেয় কে

Procfile-এ এক লাইনেই পুরো সিদ্ধান্ত

```
# Procfile
release: python manage.py migrate --noinput
web: gunicorn config.wsgi:application \
       --bind 0.0.0.0:$PORT \
       --workers 3 \
       --timeout 60 \
       --access-logfile - \
       --error-logfile -
```

প্যারামিটার	কারণ
`0.0.0.0:$PORT`	হোস্ট যে পোর্ট দেয়, সেটিতেই শুনতে হবে
`--workers 3`	আনুমানিক ২×CPU+১; ছোট ভিড়ে ৩টিই যথেষ্ট
`--timeout 60`	৬০ সেকেন্ডে উত্তর না দিলে শ্রমিক মারা যাবে (আটকে থাকা অনুরোধ নয়)
`--access-logfile -`	লগ stdout-এ — হোস্টের লগ ভিউয়ারে দেখা যাবে
`release:`	নতুন কোড চালু হওয়ার **আগে** মাইগ্রেশন

শ্রমিক বাড়ালে মেমরি বাড়ে, আর `CONN_MAX_AGE=60` মানে প্রতি শ্রমিক ডেটাবেসে সংযোগ ধরে রাখে — ৩ শ্রমিক × সংযোগ = হিসাব করে বাড়ান, ইচ্ছেমতো নয়।

শিক্ষকের নোট: "Gunicorn = অনেক শ্রমিক, প্রত্যেকে একটি অনুরোধ সামলায়; Django কোড অপরিবর্তিত থাকে।" WSGI কী — সংক্ষেপে: পাইথন অ্যাপ আর ওয়েব সার্ভারের মধ্যে চুক্তি।

বিল্ড

চার লাইনের `build.sh`

ক্রমটাই আসল — উল্টালে ডিপ্লয় ভাঙে

```
#!/usr/bin/env bash
set -euo pipefail          # যেকোনো ধাপ ব্যর্থ = থামো
python -m pip install --upgrade pip
pip install -r requirements-prod.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput
```

কেন `set -euo pipefail`

ব্যর্থ ধাপ চুপচাপ পেরিয়ে গেলে আধা-বিল্ড চালু হয় — সবচেয়ে বিভ্রান্তিকর ব্যর্থতা।

requirements-prod.txt

`-r requirements.txt` + `gunicorn` + `whitenoise` + `dj-database-url`

collectstatic আগে, migrate পরে

হ্যাশ করা ফাইল নাম দরকার; মাইগ্রেশন ব্যর্থ হলে নতুন কোড চালু হবে না, পুরোনো চলবে।

--noinput কেন

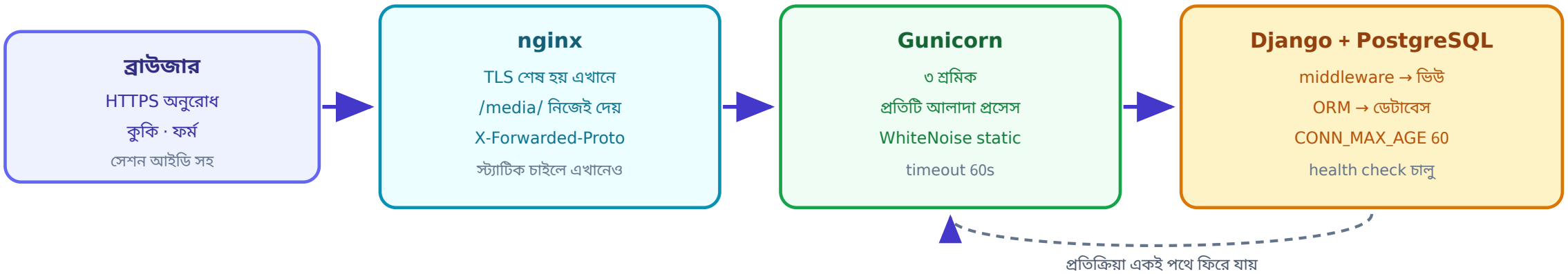
স্বয়ংক্রিয় বিল্ডে কেউ "yes" চাপবে না — প্রশ্ন করলেই বিল্ড ঝুলে থাকবে।

শিক্ষকের নোট: বলুন: "বিল্ড স্ক্রিপ্ট হলো রেসিপি — টার্মিনালে হাতে যা করেন, সেটাই লিখে রাখা।" হাতে করা ধাপ ভুলে যাওয়াই সবচেয়ে সাধারণ ডিপ্লয়-বাগ।

পথ

একটি অনুরোধের যাত্রা

ব্রাউজার → nginx → Gunicorn → Django



কেন দুটি সার্ভার: nginx ধীর ক্লায়েন্ট, TLS, ফাইল সামলায়; Gunicorn শুধু পাইথন কোড।

ভুল হলে: `SECURE_PROXY_SSL_HEADER` না থাকলে Django-র কাছে অনুরোধ http — রিডাইরেক্ট লুপ।

শিক্ষকের নোট: বোর্ডে চারটি বাক্স আঁকুন, তারপর জিজ্ঞেস করুন: "কোন ধাপে ডেটা পড়া যায়?" উত্তরে বোঝান — TLS nginx-এ শেষ, তাই nginx আর Gunicorn-এর মধ্যে প্লেইন HTTP শুধু ভেতরের নেটওয়ার্কে।

ডেটাবেস

প্রোডাকশনে PostgreSQL

তিনটি সিদ্ধান্ত: সংযোগ, মাইগ্রেশন, ডেটা

```
# prod.py — সংযোগ পুনরায় ব্যবহার
DATABASES["default"]["CONN_MAX_AGE"] = 60
DATABASES["default"]["CONN_HEALTH_CHECKS"] = True

# সার্ভারে (আসল কাজ)
createdb ims_prod
python manage.py migrate --settings=config.settings.prod
python manage.py createsuperuser --settings=config.settings.prod

# NEVER on a real deployment:
python manage.py seed_demo     # admin@ims.local + DemoPass!2026
```

CONN_MAX_AGE = 60

প্রতি অনুরোধে নতুন সংযোগ নয় — ৬০ সেকেন্ড সংযোগ ধরে রাখা; শিক্ষার্থীর তালিকা খুললে পার্থক্য বোঝা যায়।

HEALTH_CHECKS

পুরোনো সংযোগ মৃত হলে ব্যবহারের আগেই ধরা পড়ে — "server closed the connection" এরর কমে।

seed_demo নিষিদ্ধ

সবার জানা পাসওয়ার্ডের সুপারইউজার বানায় — ডেমোর জন্য ভালো, প্রোডাকশনে বিপর্যয়।

SQLite কেন নয়

এক লেখক, দুর্বল কনকারেন্সি — ৩০ শিক্ষকের মার্কস এন্ট্রিতে লক।

শিক্ষকের নোট: ডেমো ও প্রোডাকশনের ডেটাবেস আলাদা — এই সতর্কতাটি বারবার বলুন, কারণ `seed_demo` চালানো খুব সহজ।

রিহার্সাল

`deploy_check.sh` — ডিপ্লয়ের আগেই ডিপ্লয়

Gunicorn :8010, DEBUG=False, ৫টি দাবি

```
gunicorn config.wsgi:application --bind 127.0.0.1:8010 \
         --workers 2 --daemon --pid /tmp/ims-prod.pid

check "login page"               /accounts/login/     # 200
check "static css"               /static/css/ims.css  # 200
check "unknown page (404 page)"  /does-not-exist      # 404
check "workspace page anon"      /w/sunrise-model-school/  # 302

curl -sI /accounts/login/ | grep -iE \
  "x-content-type|x-frame|referrer|cross-origin"
```

৫টি প্রশ্নের উত্তর

লগইন খোলে? CSS আসে? ৪০৪ পাতা কাজ করে? অচেনা মানুষ ভেতরে ঢোকে না? হেডার আছে?

কেন লোকালে

সার্ভারে গিয়ে ভাঙার চেয়ে নিজের মেশিনে ভাঙা অনেক সস্তা।

কমDEBUG-এ যা দেখা যায়

স্ট্যাটিক ৪০৪, ৫০০ পাতার রূপ, রিডাইরেক্ট — এগুলো DEBUG=True-তে কখনো ধরা পড়ে না।

শিক্ষকের নোট: ক্লাসে চালাতে চাইলে PostgreSQL চালু থাকতে হবে — নইলে ৫০০ দেখে বিভ্রান্ত হবেন। বিকল্প: `check --deploy` আর হেডারগুলো দেখান।

চেকলিস্ট

Go-live-এর আটটি দরজা

DEPLOYMENT.md §8 থেকে বাছাই

যাচাই	না হলে
`SECRET_KEY` ৫০+ অক্ষর, হোস্টের সিক্রেট স্টোরে	কোডে লেখা কী = ফাঁস হওয়া কী
`ALLOWED_HOSTS` ও `CSRF_TRUSTED_ORIGINS`-এ আসল ডোমেইন	400 / 403, ফর্ম সাবমিট ব্যর্থ
http → https রিডাইরেক্ট কাজ করে	পাসওয়ার্ড প্লেইনে যায়
"Forgot password" একবার শুরু থেকে শেষ পর্যন্ত	SMTP ভুল — ব্যবহারকারী আটকে থাকে
`check --deploy` সার্ভারে "no issues"	অজানা ফাঁক রয়ে যায়
ব্যাকআপ চলেছে **এবং** restore একবার পরীক্ষা হয়েছে	ব্যাকআপ আছে বলেই ভরসা — ভুল ভরসা
ডেমো ডেটা মুছে (`DemoPass!2026` নেই)	যে কেউ সুপারইউজার
হোস্ট `gunicorn` চালায়, `runserver` নয়	একজন ব্যবহারকারীতেই সাইট আটকে যায়

DEPLOYMENT.md-এ লেখা আছে: "It has **not** yet been deployed to a public host" — অর্থাৎ এই ৮টি দরজা এখনো কেউ সম্পূর্ণ পেরোয়নি। সৎ খাতা রাখাই এখানেও নিয়ম।

শিক্ষকের নোট: ছাত্রদের বলুন এই ৮টি নিজের প্রজেক্টে মিলিয়ে দেখতে; যেটি "না", সেটাই পরের ক্লাসের আলোচনা।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের অ্যাপকে সার্ভারের ভঙ্গিতে চালান

প্রথম ১০ মিনিট হাতে, শেষ ১০ মিনিট প্রশ্ন

১. ডেভ কী বানান

```
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

২. ভুল মান দিয়ে চালান

প্রথমে একই ডেভ কী, পরে `ALLOWED_HOSTS="*"` — দুই রকম থামা দেখুন।

৩. সঠিক মান দিয়ে `check --deploy`

"no issues" না আসা পর্যন্ত সেটিং ঠিক করুন — সেটিং নয়, কারণ ঠিক করুন।

৪. হেডার যাচাই

`curl -sI` দিয়ে HSTS, X-Frame-Options, Referrer-Policy খুঁজুন।

- DEBUG=True রাখলে কী কী ফাঁস হয়?
- HSTS আগে চালু করলে কী বিপদ?
- মিডিয়া ফাইল কোথায় থাকবে?
- শ্রমিক সংখ্যা বাড়ালে কী বাড়ে?
- `seed_demo` প্রোডাকশনে চালালে?

শিক্ষকের নোট: দ্বিতীয় ধাপে সবাই নিজের চোখে থামতে দেখুক — এটি মুখস্থ নিয়মের চেয়ে অনেক বেশি কার্যকর।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

ডিপ্লয়মেন্টের পুরো সারাংশ

নীতি	প্রমাণ
runserver সার্ভারের জন্য নয়	`Procfile`: gunicorn, ৩ শ্রমিক
সেটিংস তিন ভাগে	`base` / `dev` / `prod`
গোপন মান পরিবেশে, কোডে নয়	`env()` · `.env.example`
ভুল মানে অ্যাপ চালুই হবে না	দুটি `RuntimeError`
ডিপ্লয়ের আগে লোকালে রিহার্সাল	`deploy_check.sh` · ৫টি চেক
ব্যাকআপ যাচাই না করলে ব্যাকআপ নয়	`ops/restore.sh` — scratch ডিবি

পরের ক্লাসে: এই সার্ভারকে **দিনের পর দিন** সচল রাখা — লগ, মনিটরিং, ব্যাকআপের সময়সূচি, আপডেট, এবং রোলব্যাক।

শিক্ষকের নোট: শেষ বাক্যটি দিয়ে পরের ক্লাসের টান তৈরি করুন — "ডিপ্লয় একবারের কাজ, চালু রাখা প্রতিদিনের কাজ।"

২৯

পরিচালনা — সার্ভার চালু রাখা

লগ পড়া · মনিটরিং · ব্যাকআপ সময়সূচি · আপডেট · রোলব্যাক · ৫০০ এরর তদন্ত

পর্ব ৬ · ক্লাস ২৯

সার্ভার চালু রাখা

লগ · অডিট · ব্যাকআপ ও রিস্টোর · মাইগ্রেশন · রোলব্যাক · মনিটরিং

IMS · ক্লাস ২৯ · স্লাইড ২০–৩৮

শুরু

ডিপ্লয় একদিনের কাজ, পরিচালনা প্রতিদিনের

কেন "চালু হয়েছে" মানে "শেষ" নয়

প্রথম দিন

কেউ ভুল করে, তবু কাজ চলে। তৃতীয় সপ্তাহে: ডেটা বাড়ে, ধীর হয়, একবার ৫০০ আসে, কেউ জিজ্ঞেস করে "গত মাসে কে মার্কস বদলেছিল?"

যে তিনটি প্রশ্ন আসবেই

১. কী ভেঙেছে? (লগ) ২. কে কী করেছিল? (অডিট) ৩. কিছু হারিয়েছে কি? (ব্যাকআপ)

আর একটি

৪. এখন ঠিক করব কীভাবে, আর ফেরাব কীভাবে? (মাইগ্রেশন ও রোলব্যাক)

ডিপ্লয়মেন্ট ক্লাসে শিখেছেন "কীভাবে চালু করতে হয়"; এই ক্লাসে শিখবেন **কীভাবে টিকে থাকে** — এবং প্রথম নিয়ম: যা মাপা হয় না, তা রক্ষা করা যায় না।

শিক্ষকের নোট: জিজ্ঞেস করুন: "আপনার অ্যাপে এই চারটি প্রশ্নের উত্তর কোথায় আছে?" বেশিরভাগের উত্তর "কোথাও নেই" — এটাই এই ক্লাসের কারণ।

ছন্দ

দৈনিক · সাপ্তাহিক · মাসিক

পরিচালনার কাজ ভাগ করে নিন — সব একদিনে নয়

কখন	কাজ	IMS-এ কী দিয়ে
প্রতিদিন	ত্রুটি দেখুন · ব্যাকআপ চলেছে কি না দেখুন	হোস্টের লগ · `ops/backup.sh` (ক্রন)
প্রতিদিন	বাণিজ্যিক অবস্থা	`manage.py billing_report`
সাপ্তাহিক	ডিস্ক, ধীর কোয়েরি, বাড়তে থাকা টেবিল	ডেটাবেস প্রদানকারীর ড্যাশবোর্ড
সাপ্তাহিক	একটি রিস্টোর পরীক্ষা (scratch ডিবি)	`ops/restore.sh`
মাসিক	নির্ভরশীল প্যাকেজ আপডেট ও `check`	`pip list --outdated` · `test` · `check --deploy`
প্রতি রিলিজে	ব্যাকআপ → মাইগ্রেশন → ধোঁয়া পরীক্ষা	`build.sh` · `tests` · `smoke_test`

নিয়ম: **প্রতিটি কাজের একটি কমান্ড থাকতে হবে** — মনে রাখার মতো কাজ নয়, চালানোর মতো কাজ।

শিক্ষকের নোট: ছোট প্রতিষ্ঠানে এই ছকটি একজনেরই কাজ — তাই যত ছোট হোক, লিখিত থাকতে হবে; ছুটিতে অন্য কেউ চালাবে।

লগ

লগ কোথায় যায়

IMS-এ কেবল কনসোল — সেটাই যথেষ্ট, যদি আপনি দেখেন

```
# base.py
LOGGING = {
  "handlers": {"console": {"class":
      "logging.StreamHandler", ...}},
  "root": {"handlers": ["console"],
           "level": env("LOG_LEVEL", "INFO")},
  "loggers": {
    "django.request":  {"level": "WARNING", ...},
    "django.security": {"level": "WARNING", ...},
  },
}

# prod.py — একই কাঠামো, তবে ফরম্যাটে
# [{asctime}] {levelname} {name} {process:d}: {message}
# # প্রসেস আইডি থাকে => কোন শ্রমিক ব্যর্থ
# # হয়েছে তা বোঝা যায়
```

stdout কেন

কনটেইনার যুগে হোস্ট stdout সংগ্রহ করে — ফাইলে লিখলে রিডিপ্লয়ে হারায়।

দুই বিশেষ লগার

`django.request` = ৫০০ ও 4xx; `django.security` = CSRF, হোস্ট, সন্দেহজনক অনুরোধ।

যা এখানে নেই

ফাইল হ্যান্ডলার নেই, Sentry নেই, APM নেই — `requirements-prod.txt`-এ কোনো মনিটরিং প্যাকেজ নেই।

প্রথম সংযোজন

`LOG_LEVEL=INFO` রাখুন; এরর খুঁজতে হোস্টের লগ ভিউয়ারে `ERROR` ফিল্টার করুন।

শিক্ষকের নোট: সৎ কথা বলুন: পেশাদার সেটআপে Sentry-জাতীয় সেবা থাকে; IMS-এ নেই। শিক্ষার্থীদের প্রশ্ন করুন — "কী হারাচ্ছি এতে?" উত্তর: পাওয়া যায় না কোন ব্যবহারকারী কোন পৃষ্ঠায় ব্যর্থ হয়েছেন।

পড়া

লগে কী খুঁজবেন

চারটি প্যাটার্ন, চারটি প্রশ্ন

যা খুঁজছেন	লগে দেখতে হবে	তারপর
৫০০ এরর	`ERROR django.request Internal Server Error: /w/…`	ট্রেসব্যাক, সময়, প্রসেস আইডি
হঠাৎ ৪০৩ ঝাঁপ	`WARNING django.request Forbidden` বারবার	ভূমিকা/স্কোপ বদল হয়েছে কি না
CSRF ব্যর্থতা	`WARNING django.security.csrf`	ডোমেইন বা প্রক্সি হেডার
ধীর অনুরোধ	দেখা যায় না — সময় মাপা হয় না	এখানেই ফাঁক: পরে যোগ করতে হবে

সোনার নিয়ম: লগে ব্যক্তিগত তথ্য নয় — `AuditLog` মডেলের ডকুমেন্টেশনেই লেখা: "never store passwords, reset tokens or secrets"।

ফাঁদ: শিক্ষকের ইমেইল বা ছাত্রের নম্বর লগে ছাপালে সেটি সবার দেখার জায়গায় চলে যায় — সম্মতি ছাড়াই।

শিক্ষকের নোট: চতুর্থ সারিটি জোর দিয়ে বলুন — "যা মাপি না, তা জানি না।" পারফরম্যান্স সমস্যা ধরার একমাত্র উপায় সময় মাপা।

অডিট

অ্যাপ্লিকেশন লগ ≠ অডিট ট্রেইল

একটি ডেভেলপারের জন্য, অন্যটি প্রতিষ্ঠানের জন্য

দিক	লগ (stdout)	AuditLog (ডেটাবেস)
প্রশ্ন	"কী ভেঙেছে?"	"কে, কখন, কী বদলাল?"
থাকে কোথায়	হোস্টের লগ সংরক্ষণে (দিন-সপ্তাহ)	ডেটাবেসে — ব্যাকআপের সঙ্গে
কে পড়ে	ডেভেলপার/অপারেটর	প্রতিষ্ঠানের অ্যাডমিন (`OWNERS`)
রেকর্ড	অগঠিত লেখা	গঠিত: `action` , `object_type` , `before` , `after` , `reason` , `ip_address`
না থাকলে	অন্ধ তদন্ত	বিশ্বাস হারানো — "আমার নম্বর কে বদলাল?"

IMS-এ ২৩ ধরনের অ্যাকশন লগ হয়: `students.create` · `attendance.correct` · `assessments.publish` · `homework.review` …

নকশার সিদ্ধান্ত: `log()` কখনো এরর তোলে না — অডিট ব্যর্থ হয়ে কাজ আটকাবে না, তবে ফাঁক থেকে যেতে পারে।

শিক্ষকের নোট: প্রশ্ন: "অডিট লেখা ব্যর্থ হলে কাজ থামানো উচিত?" ভালো আলোচনা: থামালে ব্যবহারকারী ক্ষতিগ্রস্ত, না থামালে প্রমাণ হারায় — IMS দ্বিতীয়টি বেছেছে, সচেতনভাবে।

খোঁজা

অডিট ট্রেইল পড়া

পর্দায় ফিল্টার, কোডে সাধারণ ORM

```
@workspace_view(*OWNERS)
def audit_list(request, slug):
    qs = AuditLog.objects.filter(
        workspace=request.workspace).select_related("actor")
    if request.GET.get("action"):
        qs = qs.filter(action__startswith=
                       request.GET["action"])
    if request.GET.get("actor"):
        qs = qs.filter(actor_id=request.GET["actor"])
    page = Paginator(qs, 40).get_page(
        request.GET.get("page"))
    actions = (AuditLog.objects
               .filter(workspace=request.workspace)
               .values_list("action", flat=True)
               .distinct().order_by("action"))
```

কেবল OWNERS

শিক্ষক বা ছাত্র নিজের বিরুদ্ধে লেখা রেকর্ড পড়তে পারে না — `@workspace_view(*OWNERS)`।

ইনডেক্স আগেই

`(workspace, -created_at)` আর `action`-এ ইনডেক্স — তালিকা বাড়লেও ধীর হবে না।

আজকের মাপা অবস্থা

`AuditLog.objects.count()` = ০ — কারণ `seed_demo` কোনো কাজ করে না। বাস্তবে শিক্ষক যা করবেন, তা-ই জমবে।

তদন্তের ছন্দ

অ্যাকশন দিয়ে ফিল্টার → সময় দেখুন → `before` / `after` মিলিয়ে দেখুন।

শিক্ষকের নোট: ডেমোতে ০ সারি দেখে হতাশ হবেন না — ক্লাসেই একটি মার্কস সংশোধন করুন, তারপর আবার তালিকা খুলুন: নতুন সারি হাজির।

ব্যাকআপ

ব্যাকআপ = দুইটি জিনিস

ডেটাবেস + আপলোড করা ফাইল

```
#!/usr/bin/env bash
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST="${BACKUP_DIR:-./backups}"
mkdir -p "$DEST"

pg_dump -Fc -f "$DEST/ims-$STAMP.dump"          # ডেটাবেস
tar -czf "$DEST/media-$STAMP.tar.gz" media/ \
    2>/dev/null || true                          # লোগো/ফাইল

find "$DEST" -name '*.dump'   -mtime +14 -delete
find "$DEST" -name '*.tar.gz' -mtime +14 -delete

# cron: রাত ২টায়
# 0 2 * * * BACKUP_DIR=/var/backups/ims \
#   bash /srv/ims/ops/backup.sh
```

কেন `-Fc`

কাস্টম ফরম্যাট = সংকুচিত + টেবিল বাছাই করে রিস্টোর করা যায়।

media আলাদা কেন

PostgreSQL-এ লোগো নেই — সেটি ফাইলে; ব্যাকআপ না করলে রিস্টোরে ফাঁকা লোগো।

১৪ দিন কেন

"গত সপ্তাহে ভুল হয়েছিল" ধরার জন্য যথেষ্ট; তার বেশি রাখলে ডিস্ক ভরে যায়।

ব্যাকআপ কোথায় রাখবেন না

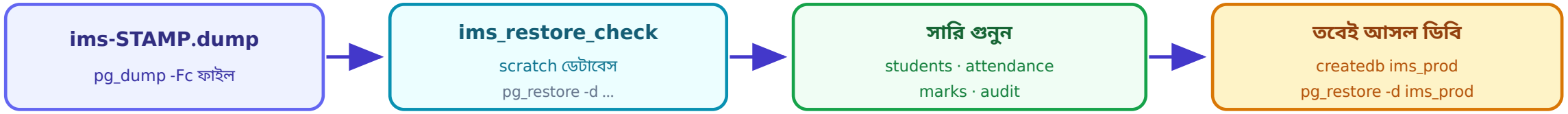
একই সার্ভারে — সার্ভার গেলে ব্যাকআপও যায়। আলাদা জায়গা বা অবজেক্ট স্টোরেজ।

শিক্ষকের নোট: প্রশ্ন করুন: "ডেটাবেসের ব্যাকআপ আছে কিন্তু media নেই — কী হারাবেন?" উত্তর: সব লোগো, আপলোড করা পড়ার উপকরণ।

রিস্টোর

আগে scratch-এ, তারপর আসল

জীবন্ত ডেটাবেসের ওপর কখনো রিস্টোর নয়



সংখ্যা মিলছে না? এখানেই থামুন — আসল ডেটাবেসে হাত দেবেন না।

> **"A backup you have never restored is not a backup."** — DEPLOYMENT.md §6। প্রতিটি রিস্টোর একই সঙ্গে ব্যাকআপের **পরীক্ষা**।

scratch ডেটাবেস রাখা যায় না বলে মুছে ফেলুন, তবে **রিস্টোরের সময় ও সারি-সংখ্যা লিখে রাখুন** — পরের বার তুলনা করার জন্য।

শিক্ষকের নোট: বলুন: "রিস্টোর শিখতে সঙ্কটের জন্য অপেক্ষা করবেন না।" মাসে একবার scratch-এ রিস্টোর করুন — ৫ মিনিটের কাজ।

মাইগ্রেশন

স্কিমা বদলানো — সবচেয়ে ঝুঁকির কাজ

ক্রম, ব্যাকআপ, আর অপরিবর্তনীয়তার নিয়ম

#	ধাপ	কেন
১	`ops/backup.sh` হাতে একবার	মাইগ্রেশন ব্যর্থ হলে ফেরার পথ থাকতে হবে
২	`makemigrations` → ফাইল পড়া	ORM যা বানিয়েছে, তা-ই তো চেয়েছিলেন?
৩	প্রথমে যোগ করুন, পরে সরান	পুরোনো কোড চলাকালীন নতুন কলাম দরকার হয়
৪	`migrate --noinput` (release step)	নতুন কোড চালু হওয়ার আগে স্কিমা তৈরি
৫	`python manage.py test` + ধোঁয়া পরীক্ষা	মাইগ্রেশনই সবচেয়ে বড় অদৃশ্য ভাঙার জায়গা

দুই ধাপের পদ্ধতি: প্রথম রিলিজে নতুন কলাম (nullable), দ্বিতীয় রিলিজে পুরোনো কলাম বাদ — এতে রোলব্যাক নিরাপদ থাকে।

ফাঁদ: `migrate` স্বয়ংক্রিয়ভাবে ডেটা ফেরায় না — কলাম মুছে ফেললে সেটি **চিরতরে** যায়।

শিক্ষকের নোট: ৩ নম্বর ধাপটি পেশাদার অভ্যাস — "expand then contract"। শিক্ষার্থীদের বলুন বড় স্কিমা বদল সবসময় দুই রিলিজে ভাগ করতে।

রোলব্যাক

"ফিরিয়ে নিন" — দুই রকম, দুই সমস্যা

কোড ফেরানো সহজ, ডেটা ফেরানো কঠিন

কোড রোলব্যাক

আগের রিলিজে ফেরানো — হোস্টের "Rollback to previous deploy"। দ্রুত, নিরাপদ, কারণ পুরোনো কোড আগেও চলেছে।

স্কিমা রোলব্যাক

`migrate app 0007` — টেবিল/কলাম ফেরে, কিন্তু নতুন রিলিজে লেখা **ডেটা ফেরে না**।

ডেটা রোলব্যাক

শুধু ব্যাকআপ থেকে। তাই রিলিজের আগে ব্যাকআপ নেওয়া = রোলব্যাকের একমাত্র পথ।

পরিস্থিতি	প্রতিক্রিয়া
নতুন পাতা ৫০০ দিচ্ছে	কোড রোলব্যাক
মার্কস ভুল দেখাচ্ছে	কোড ঠিক + অডিট দেখুন
মাইগ্রেশন অর্ধেক চলেছে	ব্যাকআপ রিস্টোর (scratch পরীক্ষা করে)
ভুল ডেটা লেখা হয়েছে অনেক আগে	অডিট `before` / `after` দিয়ে হাতে সংশোধন

নিয়ম: রোলব্যাক করার আগে **বর্তমান অবস্থার ব্যাকআপ** নিন — নইলে ভুল ঠিক করতে গিয়ে আরও ডেটা হারাবেন।

শিক্ষকের নোট: এই স্লাইডের মূল কথা: "কোডের সংস্করণ আছে, ডেটার নেই।" তাই রিলিজ-পূর্ব ব্যাকআপকে অভ্যাসে পরিণত করুন।

মনিটরিং

কী মাপবেন — ছয়টি সংখ্যা

আর IMS-এ এখন কী নেই, সেটাও স্পষ্ট

মাপ	কেন	সস্তা উপায়
uptime	সাইট খোলা আছে কি না	বাইরের পিং সেবা (৫ মিনিট পরপর)
৫০০ রেট	ব্যবহারকারী ভাঙা পর্দা দেখছে কি না	লগে `ERROR` সারি গোনা
উত্তর-সময় (p95)	ধীর হচ্ছে কি না	হোস্টের ড্যাশবোর্ড
ডেটাবেস সংযোগ	শ্রমিক × `CONN_MAX_AGE` সীমা ছাড়াচ্ছে?	ডেটাবেস প্রদানকারীর গ্রাফ
ডিস্ক	ব্যাকআপ ও `media/` ভরছে	হোস্টের অ্যালার্ট
ব্যাকআপ সফলতা	নীরবে ব্যর্থ হওয়া সবচেয়ে বিপজ্জনক	স্ক্রিপ্ট শেষে ইমেইল/cron mail

সৎ হিসাব: IMS-এ Sentry/APM/uptime বোর্ড নেই — ৬টির মধ্যে কেবল হোস্টের ডিফল্ট মেট্রিক আছে। সবচেয়ে সস্তা তিনটি আগে যোগ করুন: বাইরের পিং, লগ-অ্যালার্ট, ব্যাকআপ-ইমেইল।

শিক্ষকের নোট: "নীরব ব্যর্থতা" শব্দটি ব্যাখ্যা করুন: cron কাজ করেছে কিন্তু pg_dump ব্যর্থ — কোনো কেউ জানে না, তিন মাস পর জানা যায়।

বাণিজ্যিক

সাবস্ক্রিপশন চালানো — একটি কমান্ড

আজ সত্যি চালানো আউটপুট

```
$ python manage.py billing_report
Commercial report 2026-11-05
  active institutions : 1
  on trial            : 1
  MRR                 : ৳6,500   ARR ৳78,000
  student seats       : 10

No unpaid invoices.
```

স্ক্রিপ্ট লেখার উদ্দেশ্যই ছিল: **সকালের স্ট্যান্ড-আপে পেস্ট করা** অথবা cron-এ চালিয়ে ইমেইল করা।

কী দেখায়

সক্রিয় প্রতিষ্ঠান · ট্রায়ালে কত · MRR/ARR · মোট সিট · ৭ দিনে শেষ হওয়া ট্রায়াল · unpaid ইনভয়েস

মেয়াদ শেষে কী হয়

`SubscriptionGateMiddleware` লেখা আটকায়, পড়া চালু রাখে — ডেটা কখনো মুছে যায় না।

কে কী দেখে

অ্যাডমিন → রিনিউ পর্দায় রিডাইরেক্ট;
বাকিরা → "ask your administrator" বার্তা, কোনো বিলিং তথ্য নয়।

এখনো হাতে

অনলাইন পেমেন্ট নেই — bKash/ব্যাংক ট্রানজেকশন আইডি হাতে এন্ট্রি, ইচ্ছাকৃতভাবে।

শিক্ষকের নোট: প্রথম দিনেই `billing_report` চালান — বাস্তব সংখ্যা দেখলে "SaaS চালানো" বিমূর্ত থাকে না।

ইন্সিডেন্ট

কিছু ভেঙেছে — পাঁচ ধাপ

শৃঙ্খলা, তারপর দক্ষতা

#	ধাপ	কাজ
১	থামান ও জানান	প্রভাব বোঝা: কে আটকে, কী হারাচ্ছে। ব্যবহারকারীকে সৎ বার্তা দিন।
২	পুনরুৎপাদন	কোন URL, কোন ভূমিকা, কোন সময় — লগে সেই মুহূর্ত খুঁজুন।
৩	সীমিত করুন	দরকার হলে ফিচার বন্ধ/রোলব্যাক — ডেটা বাঁচানো আগে, কারণ পরে।
৪	ঠিক করুন + টেস্ট	প্রথমে ব্যর্থ টেস্ট (ক্লাস ২৭-এর লাল-সবুজ), তারপর সমাধান।
৫	লিখে রাখুন	কী ঘটল, কেন, কী বদলাল — পরের জন যেন একই ফাঁদে না পড়ে।

দোষ নয়, কারণ: "কে ভুল করল" নয় — "কোন নিয়ম অনুপস্থিত ছিল" এই প্রশ্নেই আসল উত্তর মেলে।

সময় লিখুন: টাইমস্ট্যাম্প ছাড়া লগ অর্থহীন — সার্ভারের সময় অঞ্চল `Asia/Dhaka` কিনা নিশ্চিত করুন।

শিক্ষকের নোট: ক্লাসে একটি কাল্পনিক ইন্সিডেন্ট বানিয়ে পাঁচ ধাপ হাঁটুন: "শিক্ষক মার্কস সংরক্ষণ করতে পারছেন না, ৫০৩ আসছে।"

৫০০ পাতা

ব্যবহারকারী যা দেখে, আর ডেটা যা ভোগে

ভদ্র বার্তা, লেনদেন অটুট — তবে একটি প্রতিশ্রুতি এখনো অপূর্ণ

```
Something went wrong on our side
The error has been written to the application
log with a reference number. No data was
lost and nothing you entered was half-saved
— every write is wrapped in a database
transaction.

If it keeps happening, contact the
administrator with the time it occurred.

[Try the dashboard again]
```

যা সত্যি

DEBUG=False-এ ডেটা ফাঁস হয় না; লেখা `transaction.atomic`-এ মোড়া, তাই অর্ধেক লেখা থাকে না।

যা এখনো সত্যি নয়

"reference number" লেখা আছে, কিন্তু কোডে এমন কোনো আইডি তৈরি হয় **না** — ভবিষ্যতের কাজ।

কেন গুরুত্বপূর্ণ

ব্যবহারকারী সময় বললে আপনি লগে খুঁজতে পারেন — আইডি থাকলে খোঁজা সেকেন্ডের কাজ হতো।

শিক্ষা

বার্তা যত সুন্দর, প্রতিশ্রুতি তত যাচাইযোগ্য হতে হবে — নইলে ভরসা ভাঙে।

শিক্ষকের নোট: এই স্লাইডটি সততার উদাহরণ হিসেবে ব্যবহার করুন — কোড আর লেখা মিলছে না, সেটি লিখে রাখাই পেশাদারির প্রথম ধাপ।

পরিবর্তন

পরিবর্তন ব্যবস্থাপনা — ছোট অভ্যাস, বড় রক্ষা

প্রতিটি রিলিজ নিজের গল্প বলুক

অভ্যাস	কেন
এক কমিট = এক পরিবর্তন	রোলব্যাক করতে হলে ছোট টুকরো ফেরানো সহজ
কমিট বার্তায় "কী + কেন"	"fix" লিখলে ছয় মাস পর কিছুই বোঝা যায় না
রিলিজ নোট: কী বদলাল, কী মাইগ্রেশন	কেউ জিজ্ঞেস করলে এক জায়গায় উত্তর
বড় বদলের আগে `--dry-run` / `sqlmigrate`	চালানোর আগে SQL দেখে নেওয়া
রিলিজের আগে ব্যাকআপ, পরে ধোঁয়া পরীক্ষা	ফেরার পথ আর প্রমাণ — দুটোই দরকার

IMS থেকে: ১৯টি কমিট, প্রতিটি একটি শ্রেণি-কাজের সীমানা; `BUILD_STATUS.md`-এ প্রমাণের খাতা।

যা এখনো নেই: CI পাইপলাইন — রিলিজের আগে স্বয়ংক্রিয় টেস্ট এখনো হাতে চালাতে হয়।

শিক্ষকের নোট: বলুন: "ভবিষ্যতের আপনি অচেনা মানুষ — তাকে চিঠি লিখে রাখুন।" কমিট বার্তা সেই চিঠি।

অনুশীলন ও আলোচনা

২০ মিনিট — পরিচালনার চার হাতে-কলমে কাজ

প্রতিটি কাজ ৫ মিনিট

১. অডিট জন্ম দিন

একটি নম্বর সংশোধন করুন → `audit_list` খুলে নতুন সারি দেখুন।

২. ব্যাকআপ নিন

`BACKUP_DIR=/tmp/bk bash ops/backup.sh` → ফাইলের আকার দেখুন।

৩. রিস্টোর পরীক্ষা করুন

scratch ডিবিতে রিস্টোর → চারটি টেবিলের সারি গুনে মিলিয়ে দেখুন।

৪. বাণিজ্যিক ছবি তুলুন

`billing_report` চালিয়ে সংখ্যাগুলো নিজের ভাষায় ব্যাখ্যা করুন।

- লগ আর অডিট — কোনটি কতদিন রাখবেন?
- ব্যাকআপ একই সার্ভারে রাখলে?
- migrate ফেরানো কি সবসময় সম্ভব?
- ব্যাকআপ নীরবে ব্যর্থ হলে বুঝবেন কীভাবে?
- ৫০০ পাতার প্রতিশ্রুতি রক্ষা করবেন কীভাবে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের উত্তর: "নতুন রিলিজে লেখা ডেটা ফেরে না — তাই ব্যাকআপই আসল রোলব্যাক।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

পরিচালনার পুরো সারাংশ

নীতি	প্রমাণ / কমান্ড
যা মাপা হয় না, রক্ষা করা যায় না	লগে ৫০০ · p95 · ব্যাকআপ সফলতা
লগ ডেভেলপারের, অডিট প্রতিষ্ঠানের	২৩ অ্যাকশন · `AuditLog` · `OWNERS`
ব্যাকআপ = ডেটাবেস + `media/`	`pg_dump -Fc` · `tar -czf` · ১৪ দিন
রিস্টোর না করা ব্যাকআপ ব্যাকআপ নয়	`ops/restore.sh` → scratch ডিবি
কোডের সংস্করণ আছে, ডেটার নেই	রিলিজ-পূর্ব ব্যাকআপ
ইন্সিডেন্টে শৃঙ্খলা, দোষ নয়	৫ ধাপ · লাল টেস্ট আগে

পরের ক্লাস (৩০): ডিপ্লয়মেন্ট পর্বের শেষ ধাপ — পূর্ণাঙ্গ রিহার্সাল, সম্পূর্ণ go-live, আর পর্ব ৭-এর সূচনা।

শিক্ষকের নোট: ক্লাস শেষে প্রত্যেককে একটি কাজ দিন: "আপনার প্রজেক্টে একটি ব্যাকআপ স্ক্রিপ্ট লিখুন এবং একবার রিস্টোর করে দেখান।"

৩০

পূর্ণাঙ্গ রিহার্সাল ও go-live

শেষ তিনটি দরজা · DNS · প্রথম ব্যবহারকারী · ফোনে পাওয়া প্রথম কল

পর্ব ৬ · ক্লাস ৩০

Go-live — চালু করার দিন

DNS · TLS · অনবোর্ডিং · ডেমো ডেটা সরানো · প্রথম ৪৮ ঘণ্টা · রোলব্যাক সিদ্ধান্ত

IMS · ক্লাস ৩০ · স্লাইড ৩৯–৫৬

শুরু

চালু করার দিন সবচেয়ে বেশি ভুল হয়

পাঁচটি বাস্তব ঝুঁকি — প্রতিটির প্রতিকার আগেই জানা

ঝুঁকি	লক্ষণ	প্রতিকার
DNS এখনো ছড়ায়নি	"সাইট খুলছে না", অথচ সার্ভার ঠিক	চালু করার আগেই রেকর্ড দিন (TTL ছোট করুন)
`ALLOWED_HOSTS` পুরোনো	হঠাৎ **400 Bad Request**	ডোমেইন + `CSRF_TRUSTED_ORIGINS` আগেই বসান
ডেমো ডেটা রয়ে গেছে	কেউ `admin@ims.local` দিয়ে ঢুকে পড়ে	যাওয়ার আগে মুছুন — চেকলিস্টে আছে
ইমেইল কাজ করছে না	পাসওয়ার্ড রিসেট "পাঠানো হয়েছে" কিন্তু আসে না	একবার নিজে "Forgot password" চালান
TLS নেই, HSTS চালু	সাইট **খুলবেই না**	TLS আগে, HSTS পরে

নিয়ম: **প্রতিটি ঝুঁকির একটি যাচাইয়ের কমান্ড থাকবে** — "দেখে নিয়েছি" নয়, "আউটপুট নিয়েছি"।

শিক্ষকের নোট: পাঁচটি ঝুঁকির মধ্যে চারটিই কনফিগারেশন, কোড নয়। শিক্ষার্থীদের বলুন: "Go-live-এর ৮০% কাজ আগের সপ্তাহের প্রস্তুতি।"

কাউন্টডাউন

D-14 থেকে D-0

আগেই করুন, যাতে শেষ দিনে কেবল দেখাশোনা করতে হয়

দিন	কাজ	প্রমাণ
D-14	আসল আয়োজক বেছে নিন · ডোমেইন কেনা · TLS	https খোলে
D-10	প্রোডাকশন সেটিংস ও `.env` ভরা	`check --deploy` → no issues
D-7	প্রথম প্রতিষ্ঠানের ডেটা তৈরি (year→term→class→subject)	স্ক্রিনশট
D-5	ডেমো ডেটা সরানো · আসল অ্যাডমিন অ্যাকাউন্ট	পুরোনো লগইন ব্যর্থ
D-3	রিস্টোর ড্রিল — scratch ডিবিতে	সারি-সংখ্যা মিলেছে
D-1	ব্যাকআপ · ধোঁয়া পরীক্ষা · ব্যবহারকারীদের জানানো	চেকলিস্টে টিক
D-0	শুধু মনিটরিং — নতুন কিছু বদলাবেন না	লগ ও ৫০০-রেট

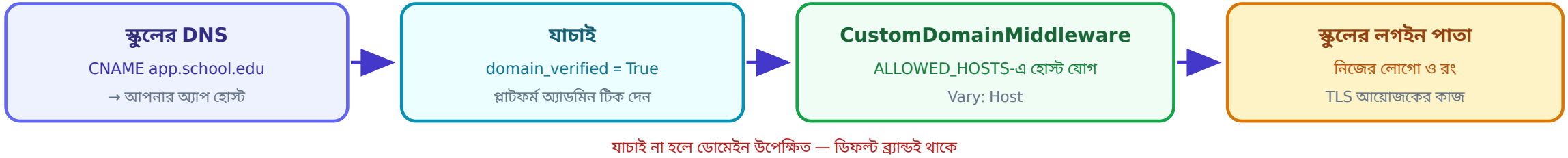
D-0-তে নতুন কিছু বদলানো **নিষিদ্ধ** — পরিবর্তন আর পর্যবেক্ষণ একসাথে করলে কোনটি সমস্যা বানাল, বোঝা যায় না।

শিক্ষকের নোট: ক্লাসে কাউন্টডাউনটি বোর্ডে লিখুন এবং প্রত্যেককে নিজের প্রজেক্টের জন্য তারিখ বসিয়ে নিতে বলুন।

DNS

কাস্টম ডোমেইন — তিন স্তর

DNS · যাচাই · রানটাইমে হোস্ট গ্রহণ



যাচাই না হলে ডোমেইন উপেক্ষিত — ডিফল্ট ব্র্যান্ডই থাকে

তিনটি টেস্টই প্রমাণ

`test_verified_domain_shows_that_institution_on_the_login_page` ·
`test_unverified_domain_is_ignored` ·
`test_domain_cannot_be_stolen_by_a_second_institution`

এক ডোমেইন, এক স্কুল

ডেটাবেসে `UniqueConstraint` — দ্বিতীয়বার লেখা যায় না।

```
if branding is not None:
    request.domain_workspace = branding.workspace
    if host not in settings.ALLOWED_HOSTS:
        settings.ALLOWED_HOSTS = (
            list(settings.ALLOWED_HOSTS) + [host])
```

শিক্ষকের নোট: প্রশ্ন: "TLS কে দেয়?" উত্তর: আয়োজক — এই অ্যাপ সার্টিফিকেট বানায় না; অ্যাপ শুধু হোস্ট গ্রহণ করে। "ডোমেইন দখল" টেস্টটির কথা বলুন: একটি ভুল রেকর্ড অন্য স্কুলের ব্র্যান্ড দেখাতে পারবে না।

কমান্ড

`sync_domains` — অনুমান বন্ধ করে

আজ সত্যি চালানো: এখনো কোনো কাস্টম ডোমেইন নেই

```
$ python manage.py sync_domains
No institution has a custom domain yet.

# ডোমেইন যোগ হলে যা ছাপে:
  app.school.edu    Sunrise Model School   verified
  old.school.edu    Sunrise Model School   PENDING

DNS records the school must create:
  CNAME  app.school.edu  ->

Environment on the server:
  ALLOWED_HOSTS=,app.school.edu
  CSRF_TRUSTED_ORIGINS=https://,https://app.school.edu
```

কেন একটি কমান্ড

DNS রেকর্ড আর দুইটি তালিকা হাতে লিখতে গিয়ে ভুল হয় — কমান্ড নকল করার মতো আউটপুট দেয়।

PENDING মানে কী

যাচাই হয়নি — এই ডোমেইন আপাতত ব্র্যান্ড পাবে না, `ALLOWED_HOSTS`-এও যাবে না।

একটি ফাঁক

`ALLOW_PREFIXES`-এ `/healthz` আছে, কিন্তু এমন কোনো URL এখনো নেই — uptime মনিটরের জন্য দরকার।

শিক্ষকের নোট: ফাঁকটি সৎভাবে দেখান: "নীতির তালিকায় আছে, বাস্তবে নেই — এভাবেই ডকুমেন্টেশন আর কোডের দূরত্ব তৈরি হয়।"

TLS

সার্টিফিকেট কার কাজ

অ্যাপ নয় — আয়োজক বা প্রক্সি

স্তর	দায়িত্ব	IMS-এ কী আছে
DNS	CNAME/A রেকর্ড	স্কুল করে · `sync_domains` যাচাইয়ে সাহায্য করে
TLS সার্টিফিকেট	আয়োজক (Render/Fly/Caddy) বা nginx + certbot	অ্যাপে নেই — ইচ্ছাকৃত
রিডাইরেক্ট ও HSTS	Django	`SECURE_SSL_REDIRECT` · `SECURE_HSTS_SECONDS`
প্রক্সি হেডার	nginx → অ্যাপ	`SECURE_PROXY_SSL_HEADER`
কুকি সুরক্ষা	Django	`Secure` · `HttpOnly` · `SameSite=Lax`

ক্রম: DNS → TLS কাজ করছে → তারপর HSTS। উল্টো করলে সাইট বন্ধ হয়ে যাবে, ফেরানো কঠিন।

যাচাই: `curl -sI https://…/accounts/login/` — `strict-transport-security` আর `x-frame-options` দেখা যায় কি না।

শিক্ষকের নোট: "সার্টিফিকেট মেয়াদ শেষ" সবচেয়ে সাধারণ আউটেজ — আয়োজক স্বয়ংক্রিয় নবায়ন করে কি না, প্রথম দিনেই দেখে রাখুন।

অনবোর্ডিং

একটি স্কুল, ২০ মিনিটে

LAUNCH_PLAYBOOK §4 — ছয় ধাপ

#	ধাপ	কোথায় / কীভাবে
১	অ্যাকাউন্ট	নিজে সাইন আপ, বা প্লাটফর্ম কনসোল → নতুন প্রতিষ্ঠান (প্ল্যান, অ্যাডমিন ইমেইল, অস্থায়ী পাসওয়ার্ড)
২	ব্র্যান্ডিং	লোগো · রং · tagline · support ইমেইল · লগইন বার্তা
৩	কাঠামো	শিক্ষাবর্ষ → টার্ম → বিভাগ → ক্লাস → বিষয় → শিক্ষক-অ্যাসাইনমেন্ট
৪	মানুষ	শিক্ষক (Teacher), কোঅর্ডিনেটর (স্কোপ সারি) — তার বেশি কিছু নয়
৫	ছাত্র	UI দিয়ে যোগ করুন — CSV আমদানি এখনো তৈরি হয়নি
৬	রিভার্সাল	একটি রিপোর্ট কার্ড একসাথে দেখুন — **এটাই তারা কিনেছে**

৬ নম্বর ধাপ বাদ দিলে ইনভয়েসের পরে "এটা তো আমরা চাইনি" শোনার ঝুঁকি — রিপোর্ট কার্ডই আসল পণ্য।

শিক্ষকের নোট: ক্লাসে ২০ মিনিটের অনবোর্ডিংটি লাইভ করুন — শিক্ষার্থীরা দেখবে কাঠামো আগে, মানুষ পরে; উল্টো করলে স্কোপ ভাঙে।

সেটআপ

ব্র্যান্ডিং, স্কোপ ও সিট সীমা

তিনটি জিনিস যা প্রথম দিনেই ঠিক হতে হবে

ব্র্যান্ডিং

`display_name` · `tagline` · `logo` · `primary_color` · `login_message` · `support_email` · `support_phone`

কোঅর্ডিনেটর স্কোপ

প্রতি বিভাগ বা ক্লাসে একটি সারি — "`CoordinatorScope`"। না দিলে সে কিছুই দেখে না, যা ভুল নয় — নিরাপদ।

সিট সীমা

ইনভয়েস নয় — **আসল ছাত্র সারি** গুনে প্ল্যান-সীমা প্রয়োগ হয়, সার্ভারে।

কে কী বদলাতে পারে

ডোমেইন ও প্ল্যান: কেবল প্লাটফর্ম অ্যাডমিন। নিজের ব্র্যান্ডিং ও লোক: প্রতিষ্ঠান অ্যাডমিন।

কেন ডোমেইন owner-only: একটি ভুল ডোমেইন অন্য প্রতিষ্ঠানের লগইন পাতায় ভুল ব্র্যান্ড দেখাবে — লিক, যদিও ডেটা নয়।

শিক্ষকের নোট: সিট সীমা প্রসঙ্গে বলুন: "বিলিং সত্য বলছে কি না, তা ডেটাবেস গুনেই ঠিক হয় — কাগজ নয়।"

পরিচ্ছন্নতা

ডেমো ডেটা সরানো — বিপদের এক নম্বর জায়গা

`seed_demo` সবার জানা পাসওয়ার্ড বানায়

```
# যা সত্যিই বিপজ্জনক
admin@ims.local         # সুপারইউজার + প্রতিষ্ঠান অ্যাডমিন
DemoPass!2026           # প্রতিটি ডেমো অ্যাকাউন্টের পাসওয়ার্ড
sunrise-model-school    # ডেমোর প্রতিষ্ঠান

# সরানোর ধাপ (নিজের দায়িত্বে, আগে ব্যাকআপ)
python manage.py shell
>>> from apps.accounts.models import User
>>> User.objects.filter(email__endswith="@ims.local").delete()
>>> User.objects.filter(is_superuser=True,
...     email="admin@ims.local").delete()

# আসল অ্যাডমিন
python manage.py createsuperuser --settings=config.settings.prod
```

যাচাই

পুরোনো লগইন চেষ্টা করুন — ব্যর্থ হওয়া **উত্তর**, অনুমান নয়।

ডেমো ডেটা মুছলে কী ভাঙে

আসল কাঠামো (বর্ষ, ক্লাস, বিষয়) নিজে বানাতে হবে — তাই আগে দেখে নিন কোনটি ডেমো, কোনটি আসল।

প্রথম দিনের সবচেয়ে বড় ভুল

"পরে সরাব" — পরে আর কেউ মনে রাখে না। চেকলিস্টে এটি থাকার কারণই এই।

শিক্ষকের নোট: `delete()` চালানোর আগে অবশ্যই ব্যাকআপ — ক্লাসেই এই শৃঙ্খলা অভ্যাস করান। আরও নিরাপদ: purging না করে নতুন প্রোডাকশন ডেটাবেসে শুধু আসল প্রতিষ্ঠানটি রাখা।

ইমেইল

পাসওয়ার্ড রিসেট — একবার নিজে চালান

সবচেয়ে বেশি আসা সাপোর্ট টিকেট

```
# dev / sqlite: কনসোলে ছাপে
EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend
# prod: SMTP (Brevo / Mailgun / Gmail app password)
EMAIL_HOST=smtp-relay.brevo.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=…
EMAIL_HOST_PASSWORD=…
DEFAULT_FROM_EMAIL=IMS

# যাচাই: /accounts/password-reset/ খুলে নিজের
# ইমেইল দিন, লিংকটি মেইলে আসা চাই
```

কনসোল ব্যাকএন্ডে প্রোডাকশনে

মেইল কোথাও যায় না — শুধু সার্ভারের লগে ছাপে। ব্যবহারকারী চিরকাল অপেক্ষা করবে।

SPF/DKIM

সেবা প্রদানকারীর ডোমেইন যাচাই না করলে মেইল স্প্যামে যাবে — "লিংক আসেনি" এররটি এখানেই।

নীরবতা নিয়ম

অ্যাকাউন্ট আছে কি নেই — উত্তর একই। তবু ব্যবহারকারী ভাবেন "মেইল এল না"; তাই সাপোর্ট স্ক্রিপ্টে এটি আগেই বলুন।

টোকেনের গোপনীয়তা

`SECRET_KEY` বদলালে পুরোনো রিসেট লিংক অচল — নিরাপত্তার দাম, জানিয়ে রাখুন।

শিক্ষকের নোট: ক্লাসে কনসোল ব্যাকএন্ডে রিসেট চালিয়ে টার্মিনালে লিংক দেখান — তারপর বলুন, প্রোডাকশনে ঠিক এই লিংকটাই মেইলে যাবে।

D-0

Go-live দিনের রানবুক

H-2 থেকে H+2 — যা করবেন, যে ক্রমে

সময়	কাজ	প্রমাণ
H-2	ব্যাকআপ নিন · `migrate` চলেছে কি না দেখুন	dump ফাইলের আকার
H-1	`tests` + `smoke_test` + `check --deploy`	তিনটি আউটপুট
H-0	রিলিজ · লগইন · একটি রিপোর্ট কার্ড · একটি CSV	স্ক্রিনশট
H+0.5	লগে `ERROR` আছে কি না	গোনা সংখ্যা
H+1	প্রথম ব্যবহারকারীকে ফোন/সরাসরি সাহায্য	একটি সমাধান হওয়া সমস্যা
H+2	ব্যাকআপ আবার · বাণিজ্যিক রিপোর্ট	`billing_report`

রানবুকে **"নতুন কিছু বদলাবেন না"** লাইনটি রাখুন — আজকের কাজ পর্যবেক্ষণ, উন্নয়ন নয়।

শিক্ষকের নোট: এই ছকটি প্রিন্ট করে দেওয়ার মতো — শিক্ষার্থীরা নিজের প্রকল্পের go-live-এ ব্যবহার করবে।

৪৮ ঘণ্টা

প্রথম দুই দিনে কী দেখবেন

সংখ্যা, গুজব নয়

সংকেত	স্বাভাবিক	উদ্বেগের
৫০০ এরর	০ (এক-দুই হলে দেখে নিন)	প্রতি ঘণ্টায় একাধিক
৪০৩	প্রথম দিনে কিছু — নতুন ভূমিকা শিখছে	একই ব্যবহারকারীর জন্য বারবার
লগইন ব্যর্থতা	প্রথম ঘণ্টায় কয়েকটি (পাসওয়ার্ড ভুল)	একটি অ্যাকাউন্টে কয়েকবার — লকআউট নেই
ধীর পাতা	বড় রিপোর্ট ধীর	ড্যাশবোর্ডও ধীর
সাপোর্ট কল	"কীভাবে..." ধরনের	"আমার ডেটা ভুল" ধরনের

যেকোনো "ডেটা ভুল" কল = সাথে সাথে **অডিট ট্রেইল** খুলুন — কার, কখন, `before` / `after`। এটি IMS-এর সবচেয়ে দামি সরঞ্জাম।

শিক্ষকের নোট: বলুন: "সাপোর্ট কলের ধরনই বলে দেয় সাইট স্বাস্থ্যকর কি না — 'কীভাবে' ভালো, 'ভুল দেখাচ্ছে' খারাপ।"

সিদ্ধান্ত

কখন রোলব্যাক করবেন

তিনটি ট্রিগার — আগেই ঠিক করে রাখুন

১. ডেটা লিখতে বাধা

শিক্ষক মার্কস/উপস্থিতি সংরক্ষণ করতে পারছেন না — এটি ব্যবসার ক্ষতি, এক ঘণ্টাও চলবে না।

২. নিয়ম ভাঙছে

এক ভূমিকা অন্যের ডেটা দেখছে — নিরাপত্তা ঘটনা; ফেরানোই সঠিক উত্তর, "পরে ঠিক করব" নয়।

৩. ৫০০ ছড়িয়ে পড়ছে

একই এরর অনেক ব্যবহারকারীর কাছে — আগের রিলিজে ফিরে লগ পড়া শুরু করুন।

নয়: "কেউ অভিযোগ করেছে" বা "পর্দায় রং অন্য রকম" — এগুলো রিলিজ বন্ধ করার কারণ নয়, তালিকা করার।

আগে: রোলব্যাকের আগেও ব্যাকআপ — নতুন রিলিজে লেখা ডেটা ফেরার একমাত্র পথ।

শিক্ষকের নোট: সিদ্ধান্ত আগে নেওয়ার কারণ: সংকটের মুহূর্তে সবাই চাপে থাকে, তখন নতুন নীতি ঠিক করা যায় না।

সাপোর্ট

ব্যবহারকারীকে কী বলবেন

সৎ, সংক্ষিপ্ত, নির্দিষ্ট — তিনটি নিয়ম

যা বলবেন না	যা বলবেন
"সার্ভারে সমস্যা হচ্ছে"	"মার্কস সংরক্ষণ এখন কাজ করছে না — ২০ মিনিটে জানাচ্ছি"
"আপনার ব্রাউজারে সমস্যা"	"আমরা দেখছি; আপনি কোন পর্দায় ছিলেন?"
"ডেটা হারাবে না" (না জেনে)	"ডেটা অটুট আছে কি না ৫ মিনিটে নিশ্চিত করছি"
"শীঘ্রই ঠিক হবে"	"আগামী ৩০ মিনিটে আপডেট দেব"

ভরসা একবারে ভাঙে, একবারে তৈরি হয় না — প্রতিটি সৎ আপডেট জমা হয়।

```
সাপোর্ট টিকেটের ছক

১. কে / কোন প্রতিষ্ঠান
২. কোন পর্দা + URL
৩. কখন (ঘড়ির সময়)
৪. কী আশা করেছিলেন
৫. কী ঘটেছে
৬. স্ক্রিনশট
৭. জরুরি? (ডেটা বন্ধ / নিয়ম ভাঙা)

→ ২ ও ৩ নম্বর দিয়েই লগে খোঁজা শুরু
```

শিক্ষকের নোট: ক্লাসে দুটি উদাহরণ লিখুন — একটি ভালো টিকেট, একটি খারাপ ("সাইট কাজ করছে না") — পার্থক্য নিজেরাই বলবে।

এরপর

প্রথম সপ্তাহের পরের তালিকা

যা বাকি, তা লেখা আছে — লুকোনো নয়

কাজ	কেন এখন
uptime মনিটর + `/healthz`	নীরব ডাউনটাইম ধরার একমাত্র উপায়
লগ-অ্যালার্ট (৫০০ হলে ইমেইল)	ব্যবহারকারীর আগে জানুন
CI: প্রতিটি পুশে `test`	হাতে চালানো ভুলে যায়
রেট লিমিট (লগইনে)	এখন কোনো থ্রটল নেই
CSV আমদানি	অনবোর্ডিংয়ের সবচেয়ে ধীর ধাপ
ব্যাকআপ কনসোল	এখন শুধু `ops/` স্ক্রিপ্ট

BUILD_STATUS-এ লেখা বাকি কাজ

invitation workflow · platform console · backup & restore console · security attack suite

নিয়ম

প্রতিটি নতুন কাজের সঙ্গে একটি টেস্ট — নইলে তালিকা আরও লম্বা হবে, আস্থা বাড়বে না।

শিক্ষকের নোট: "সফটওয়্যার কখনো শেষ হয় না — শুধু দায়িত্ব বদলায়" এই বাক্য দিয়ে পর্ব ৭-এর দিকে নিয়ে যান।

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের go-live রানবুক লিখুন

দলবদ্ধ কাজ, তারপর ভাগাভাগি

১. D-14 → D-0 ছক
নিজের প্রকল্পের জন্য তারিখ বসান — কোনটি আজই করা যায়?

২. রোলব্যাক ট্রিগার
তিনটি ট্রিগার নিজের ভাষায় লিখুন — সংখ্যা দিয়ে।

৩. ডেমো ডেটা তালিকা
কোন অ্যাকাউন্ট, কোন প্রতিষ্ঠান, কোন ফাইল — লিখে ফেলুন।

৪. সাপোর্ট ছক
৭ প্রশ্নের টিকেট ছকটি নিজের টুলে বসান (ইশিখন/ইমেইল/শীট)।

- D-0-তে কেন কিছু বদলাবেন না?
- TLS আগে না HSTS আগে?
- ডেমো ডেটা রয়ে গেলে কী হয়?
- "আমার ডেটা ভুল" — প্রথম কী দেখবেন?
- রোলব্যাকের আগে কী নেবেন?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: অডিট ট্রেইল — কার, কখন, before/after। এটি মুখস্থ করিয়ে দিন।

পর্ব ৬ সমাপ্তি

তিন ক্লাসে যা শেখা হলো

৫৬ স্লাইড · ৩ ক্লাস · ৯টি যাচাইযোগ্য প্রমাণ

ক্লাস	মূল ধারণা	প্রমাণ
২৮ · সার্ভারে তোলা	prod সেটিংস · চলক · Gunicorn · WhiteNoise	`check --deploy` → no issues · দুই `RuntimeError`
২৯ · চালু রাখা	লগ · অডিট · ব্যাকআপ/রিস্টোর	`billing_report` (MRR ৳6,500) · `AuditLog` ০ সারি
৩০ · Go-live	DNS · TLS · অনবোর্ডিং · ৪৮ ঘণ্টা	`sync_domains` · ৩টি ডোমেইন টেস্ট · ডেমো-মুক্তি

সৎ ফাঁক: `/healthz` নেই · uptime/Sentry নেই · CI নেই · রেট লিমিট নেই · ৫০০ পাতার "reference number" অপূর্ণ

পর্ব ৭-এ এই ফাঁকগুলোই **রক্ষণাবেক্ষণের দায়িত্ব** হয়ে ফিরবে — কোড বাড়লে ঝুঁকি বাড়ে।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন তিনটি ফাঁক বেছে নিয়ে লেখার জন্য — পরের পর্বের আলোচনা সেখান থেকেই শুরু হবে।

৩১

পর্ব ৭ — রক্ষণাবেক্ষণ

কোড বাড়ানো · রিফ্যাক্টর · ডেপেন্ডেন্সি আপডেট · ডেটা বৃদ্ধি · টিম ও হাতবদল